TP01_graph

2023-10-11

```
library(tidyverse)
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✔ dplyr     1.1.2     ✔ readr     2.1.4
## ✔ forcats   1.0.0     ✔ stringr   1.5.0
## ✔ ggplot2   3.4.3     ✔ tibble    3.2.1
## ✔ lubridate 1.9.2     ✔ tidyr     1.3.0
## ✔ purrr     1.0.2
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(jsonlite)
```

```
##
## Attaching package: 'jsonlite'
##
## The following object is masked from 'package:purrr':
##
##     flatten
```

1.1.1.1 Choix des identifiants de référence 5keyword uniprot ou goterm)

```
uniprot <- read_tsv("../downloads/uniprotkb_proteome_UP000000625_2023_10_11.tsv.gz", show_col_types = F)
uniprot %>% head
```

Nombre de lignes attendu : 4403

```
dim(uniprot) # 4403 ok
```

```
## [1] 4403   10
```

A partir de ce tibble, il s'agit d'extraire les bnumbers de la colonne Gene names (ordered locus). La fonction str_extract permet d'extraire un pattern à partir d'une expression régulière (ici, ce sera 'b\d+') d'une chaîne de caractères.

```
mapping <- uniprot %>%
  select(Entry, names = `Gene Names`) %>%
  mutate(bnumber=str_extract(names, 'b\\d+')) %>%
  select(bnumber, uniprotID=Entry) %>%
  filter(!is.na(bnumber)) %>% # 2023 → P0DQD7 and P0A6D5 are lost (no bnumber)
  arrange(bnumber)
mapping %>% head
```

```
dim(mapping)
```

```
## [1] 4401    2
```

utile pour recup les identifiants

1.1.1.2 Reformatage des données

A présent, il s'agit de générer la liste des protéines associées à chaque mot-clé (colonne Keywords). Pour cela, on ne gardera que les colonnes Entry et Keywords, une jointure avec mapping va permettre de faire le mapping avec les bnumbers. La fonction separate_rows va nous servir à découper la colonne Keywords et répartir les mots-clés sur des lignes du tibble

```
keywords = uniprot %>%
  select(uniprotID=Entry, keyword=Keywords) %>%
  right_join(mapping, by="uniprotID") %>% # right join to remove those without bnumber
  separate_rows(keyword, sep=';') %>%
  select(bnumber, keyword) %>% #groupby par le mem mot clef
  arrange(bnumber)
keywords %>% head
```

```
dim(keywords)
```

```
## [1] 27506     2
```

Nombre de lignes attendu : 27506

Maintenant, il ne s'agit plus que de regrouper les bnumbers par mot-clé :

on prend que les 2 col qui nous intererss et a on veut a chaque bnumer recup tous les mots clef et divise col keyword pour associe a tous les bnumber

```
ref_sets = keywords %>%
  group_by(keyword) %>%
  summarise(count=n(), elements = list(bnumber)) %>%
  ungroup %>%
  filter(count>1)  %>%
  select(id=keyword, desc=count, elements)
ref_sets
```

Pour utiliser "facilement" ces données avec python, nous allons sauvegarder le tibble au format JSON avec la library library(jsonlite) :

///# output : /## Attaching package: 'jsonlite' /## The following object is masked from 'package:purrr': /## flatten

```
# exportation
ref_sets %>%
  toJSON %>%
  write("../ref.sets/uniprot.keywords.sets.json")
```

Aperçu du contenu :

```
jq . "../ref.sets/uniprot.keywords.sets.json" | head -15
```

```
## [
##   {
##     "id": "2Fe-2S",
##     "desc": 26,
##     "elements": [
##       "b0286",
##       "b0724",
##       "b0775",
##       "b0872",
##       "b0873",
##       "b1392",
##       "b1654",
##       "b1684",
##       "b1802",
##       "b1803",
```

InterPro

refaire comme pour avec keywords pour interpro ;

```
interpro = uniprot %>%
  select(uniprotID = Entry, keyword = InterPro) %>%
  right_join(mapping, by = "uniprotID") %>%
  separate_rows(keyword, sep = ';') %>%
  filter(!keyword=='') %>% # si on regarde la col interpro, vide a la fin. donc va etre aussi enregistre comme ''
vide !!! il faut le virer sinon pourri script python
  select(bnumber, keyword) %>%
  arrange(bnumber)

interpro %>% head
```

```
dim(interpro) #20037 sans vide 15k
```

```
## [1] 15636     2
```

Maintenant, il ne s'agit plus que de regrouper les bnumbers par mot-clé :

on prend que les 2 col qui nous intererss et a on veut a chaque bnumer recup tous les mots clef et divise col keyword pour associe a tous les bnumber

```
ref_sets_inter = interpro %>%
  group_by(keyword) %>%
  summarise(count=n(), elements = list(bnumber)) %>%
  ungroup %>%
  filter(count>1)  %>%
  select(id=keyword, desc=count, elements)
ref_sets_inter
```

Pour utiliser ces données avec python, nous allons sauvegarder le tibble au format JSON avec la library library(jsonlite) :

```
# exportation
ref_sets_inter %>%
  toJSON %>%
  write("../ref.sets/uniprot.interpro.sets.json")
```

Aperçu du contenu :

```
jq . "../ref.sets/uniprot.interpro.sets.json" | head -15
```

```
## [
##   {
##     "id": "IPR000014",
##     "desc": 14,
##     "elements": [
##       "b0400",
##       "b0619",
##       "b1201",
##       "b1285",
##       "b1323",
##       "b1341",
##       "b1489",
##       "b2067",
##       "b2219",
##       "b2869",
```

GO

refaire comme pour avec keywords pour Go ;

```
names(uniprot)
```

```
##  [1] "Entry"             "Reviewed"          "Entry Name"
##  [4] "Protein names"     "Gene Names"        "Organism"
##  [7] "Length"            "Gene Ontology IDs" "InterPro"
## [10] "Keywords"
```

```
go = uniprot %>%
  select(uniprotID = Entry, keyword = 'Gene Ontology IDs') %>%
  right_join(mapping, by = "uniprotID") %>%
  separate_rows(keyword, sep = '; ') %>%
  filter(!keyword=='') %>% # si on regarde la col interpro, vide a la fin. donc va etre aussi enregistre comme ''
vide !!! il faut le virer sinon pourri script python

  select(bnumber, keyword) %>%
  arrange(bnumber)

go %>% head
```

```
dim(go) #22874
```

```
## [1] 22430     2
```

Maintenant, il ne s’agit plus que de regrouper les bnumbers par mot-clé :

on prend que les 2 col qui nous intererss et a on veut a chaque bnumer recup tous les mots clef et divise col keyword pour associe a tous les bnumber

```
ref_sets_go = go %>%
  group_by(keyword) %>%
  summarise(count=n(), elements = list(bnumber)) %>%
  ungroup %>%
  filter(count>1)  %>%
  select(id=keyword, desc=count, elements)
ref_sets_go
```

Pour utiliser “facilement” ces données avec python, nous allons sauvegarder le tibble au format JSON avec la library library(jsonlite) :

///# output : /## Attaching package: ‘jsonlite’ /## The following object is masked from ‘package:purrr’: /## flatten

```
# exportation
ref_sets_go %>%
  toJSON %>%
  write("../ref.sets/uniprot.go.sets.json")
```

Aperçu du contenu :

```
jq . "../ref.sets/uniprot.go.sets.json" | head -15
```

```
## [
##   {
##     "id": "GO:0000014",
##     "desc": 4,
##     "elements": [
##       "b0228",
##       "b0397",
##       "b0398",
##       "b2761"
##     ]
##   },
##   {
##     "id": "GO:0000018",
##     "desc": 4,
##     "elements": [
```

TU

pour faire TU: 1/ charger all tu.txt -> vire Object ID, separer la colonne ‘Genes of transcription unit’ 2/ charger all genes -> recup bnumber (correspondence avec la liste de geneTU; et faire jonction 3/ exporter ## dL TU

```
TU <- read_tsv("../downloads/all-transcription-units.txt", show_col_types = F)
TU %>% head
```

```
# verifier la diff entre object id et Site
(TU$Site == TU$`Object ID`) %>% as.numeric() %>% sum(na.rm=TRUE)
```

```
## [1] 3690
```

```
which(is.na(TU$Site))
```

```
## [1]  336 1085 1923 1950 2055 2072 2810
```

```
which(is.na(TU$`Object ID`))
```

```
## [1]  336 1085 1923 1950 2055 2072 2810
```

```
# qui sont les de nan ?
TU %>% filter(is.na(Site))
```

```
names(TU)
```

```
## [1] "Site"                       "Left"
## [3] "Right"                      "Strand"
## [5] "Genes of transcription unit" "Object ID"
```

```
# check unque
dim(TU) #3697
```

```
## [1] 3697    6
```

```
length(unique(TU$`Object ID`)) #3691 or  3697 ! DONC DOUBLON
```

```
## [1] 3691
```

```
length(unique(TU$Site)) #3691
```

```
## [1] 3691
```

SEPARE LES GENETU, UNE LIGNE PAR gene TU

```
# Separation des nom des genes
TU_reshaped = TU %>% filter(!is.na(Site)) %>%
  select(ecocyID = Site,
         gene = 'Genes of transcription unit',
         Right = "Right",
         Left = "Left",
         Strand = "Strand" ) %>%
  separate_rows(gene, sep = ' // ') %>%
  filter(!gene=='') # si on regarde la col interpro, vide a la fin. donc va etre aussi enregistre comme '' vide !
!! il faut le virer sinon pourri script python

TU_reshaped %>% head
```

```
dim(TU_reshaped) #6586
```

```
## [1] 6586    5
```

```
# Check na
TU_reshaped %>% filter(is.na(ecocyID)) #%>% length
```

```
length(unique(TU_reshaped$gene)) # 4493
```

```
## [1] 4493
```

```
length(unique(TU_reshaped$ecocyID)) # 3690
```

```
## [1] 3690
```

```
# Verifie les doublons
doublons = TU_reshaped [duplicated(TU_reshaped),]
# Vérifiez si des doublons existent
if (nrow(doublons) > 0) {
  cat("Il y a des doublons dans le dataframe.\n")
  print(doublons)
} else {
  cat("Aucun doublon n'a été trouvé dans le dataframe.\n")
}
```

```
## Aucun doublon n'a été trouvé dans le dataframe.
```

Recherche correcpondance dans liste gene number

```
all_genes <- read_tsv("../downloads/All-instances-of-Genes-in-Escherichia-coli-K-12-substr.-MG1655.txt", show_col
_types = F)
all_genes %>% head
```

```
# Separe Names et recup b avec
all_genes_reshaped <- all_genes %>%
  select(ecocyID = 'Transcription Units', gene = Genes, extracted_terms = Names) %>%
  separate_rows(extracted_terms, sep = ' // ') %>%
  filter(!gene == '') %>%
  mutate(bnumber = str_extract(extracted_terms, "b\\d+")) %>%
  filter(!is.na(bnumber)) %>%
  select(-extracted_terms) %>%
  separate_rows(ecocyID, sep = ' // ')
# si on regarde la col interpro, vide a la fin. donc va etre aussi enregistre comme '' vide !!! il faut le virer
sinon pourri script python
all_genes_reshaped %>% head
```

```
dim(all_genes_reshaped %>% unique) #6665
```

```
## [1] 6665    3
```

```
# check na
all_genes_reshaped %>% filter(is.na(bnumber))
```

```
length(unique(all_genes_reshaped$gene)) # 4489 (tu 4493)
```

```
## [1] 4489
```

```
length(unique(all_genes_reshaped$ecocyID)) # 3635 (tu 3690)
```

```
## [1] 3635
```

```
length(unique(all_genes_reshaped$bnumber)) # 4548
```

```
## [1] 4548
```

creer file

```
# Jointure entre les 2 tables, TU et all genes sur geneTU,

tus_to_export <- all_genes_reshaped %>%
  left_join(TU_reshaped, by = c("gene","ecocyID")) %>% filter(!is.na(bnumber)) %>% mutate(desc = paste(Strand, Le
ft, Right, sep = " , "))
# car doublons; mais la combinaison des 2 est unique !!!!
tus_to_export %>% head()
```

```
dim(tus_to_export) #6666 avec na 6528 sans na et 6665 dans all gene
```

```
## [1] 6666    7
```

```
# Check na
tus_to_export %>% filter(is.na(bnumber)) #%>% length
```

Pour utiliser ces données avec python, nous allons sauvegarder le tibble au format JSON avec la library library(jsonlite) :

///# output : /## Attaching package: ‘jsonlite’ /## The following object is masked from ‘package:purrr’: /## flatten

```
# exportation
tus_to_export %>%
  toJSON %>%
  write("../ref.sets/ecocys.tu.sets.json")
```

Aperçu du contenu :

```
jq . "../ref.sets/ecocys.tu.sets.json" | head -15
```

```
## [
##   {
##     "ecocyID": "TU00512",
##     "gene": "G0-16636",
##     "bnumber": "b4759",
##     "Right": 1992117,
##     "Left": 1992180,
##     "Strand": "-",
##     "desc": "- , 1992180 , 1992117"
##   },
##   {
##     "ecocyID": "TU0-13901",
##     "gene": "G7686",
##     "bnumber": "b3241",
##     "Right": 3389337,
```

EXPORT EN CSV pour neo4j

```
# pour les noeuds
tus_to_export %>% unique %>% write_csv("../neo4j.import/ecocyc.tu.csv")


# pour les relations connextion avec gene
tus_to_export %>% select(c(ecocyID,bnumber)) %>% unique %>% write_csv("../neo4j.import/ecocyc.tu.genes.csv")
```

Pathway

pour faire pathway: 1/ charger pathway.txt -> vire Object ID, separer la colonne ‘Genes of pathway’ utiliser all genes pour correspondances -> recup bnumber (correspondence avec la liste de geneTU; et faire jonction 3/ exporter

dL pathway

```
pathway <- read_tsv("../downloads/All-instances-of-Pathways-in-Escherichia-coli-K-12-substr.-MG1655.txt", show_co
l_types = F)
pathway %>% head
```

quick explore

```
# qui sont les de nan ?
pathway %>% filter(is.na(Pathways))
```

```
names(pathway)
```

```
## [1] "Pathways"         "Genes of pathway" "Object ID"        "Common-Name"
```

-> utilisation variables common names pour desc

```
# check unque
dim(pathway) # 438
```

```
## [1] 438   4
```

SEPARE LES GENES implique pathway, UNE LIGNE PAR gene pathway

```
# Separation des nom des genes, filter(!is.na(Site)) %>% pas necessaires
pathway_reshaped = pathway %>%
  select(ecocyID = Pathways,
         gene = "Genes of pathway",
         desc = "Common-Name") %>%
  separate_rows(gene, sep = ' // ') %>%
  filter(!gene=='') # enregistre le vide a la fin. donc va etre aussi enregistre comme '' vide !!! il faut le vir
er sinon pourri script python

pathway_reshaped %>% head
```

```
dim(pathway_reshaped) #2849
```

```
## [1] 2849    3
```

```
# Check na
pathway_reshaped %>% filter(is.na(ecocyID)) #ok
```

```
# check unique
length(unique(pathway_reshaped$gene)) # 1156
```

```
## [1] 1156
```

```
length(unique(pathway_reshaped$ecocyID)) # 438
```

```
## [1] 438
```

```
# Verifie les doublons
doublons2 <- pathway_reshaped[duplicated(pathway_reshaped[c("ecocyID", "gene")]), ] # Vérifier si des doublons ex
istent
if (nrow(doublons2) > 0) {
  cat("Il y a des doublons dans le dataframe.\n")
  print(doublons2)
} else {
  cat("Aucun doublon n'a été trouvé dans le dataframe.\n")
}
```

```
## Aucun doublon n'a été trouvé dans le dataframe.
```

une ligne = un couple pathway/gene

Recherche correcpondance dans liste gene number dans all gene (bd ecocyd inclu)

j ai deja recupere les bnumber a partir de all_genes

```
all_genes%>% head
```

```
 all_genes_reshaped %>% head
```

```
dim(all_genes) #4523
```

```
## [1] 4523    6
```

```
dim(all_genes_reshaped) #6666
```

```
## [1] 6666    3
```

```
dim(all_genes_reshaped %>% unique) #6665
```

```
## [1] 6665    3
```

creer fichiers

```
# Jointure entre les 2 tables, pathway et all genes. je veux recuperer la colonne bnumber

pathways_to_export = left_join(pathway_reshaped, all_genes_reshaped %>% select(gene, bnumber), by = "gene")  %>%
filter(!is.na(bnumber)) %>%
  distinct()
```

```
## Warning in left_join(pathway_reshaped, all_genes_reshaped %>% select(gene, : Detected an unexpected many-to-ma
ny relationship between `x` and `y`.
## i Row 1 of `x` matches multiple rows in `y`.
## i Row 43 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship =
##   "many-to-many"` to silence this warning.
```

```
# car doublons; mais la combinaison des 2 est unique !!!!
pathways_to_export %>% head()
```

```
# Check duplicated
dim(pathways_to_export) # 4936 avec na /4932 sans na et 6665 dans all gene
```

```
## [1] 2863    4
```

```
dim(pathways_to_export %>% unique) #2863
```

```
## [1] 2863    4
```

```
length(unique(pathways_to_export$bnumber))
```

```
## [1] 1165
```

expected 2849 (cf patways dimension). need remove duplicates -> 2863

```
# Check the difference between the 2 (why 2849 and 2863 row ). Duplicates?
doublons22 <- pathways_to_export[duplicated(pathways_to_export[c("ecocyID", "gene")]), ] # Vérifier si des doublo
ns existent
if (nrow(doublons22) > 0) {
  cat("Il y a des doublons dans le dataframe.\n")
  print(doublons22)
} else {
  cat("Aucun doublon n'a été trouvé dans le dataframe.\n")
}
```

```
## Il y a des doublons dans le dataframe.
## # A tibble: 18 × 4
##    ecocyID             gene    desc                                    bnumber
##    <chr>               <chr>   <chr>                                   <chr>
##  1 GALACTCAT-PWY       EG20049 D-galactonate degradation               b4477
##  2 GALACTCAT-PWY       GB4478  D-galactonate degradation               b4478
##  3 ECASYN-PWY          G7800   enterobacterial common antigen biosynthe… b4405
##  4 ECASYN-PWY          G7800   enterobacterial common antigen biosynthe… b4481
##  5 PWY0-521            G7724   fructoselysine and psicoselysine degrada… b3372
##  6 PWY0-521            G7724   fructoselysine and psicoselysine degrada… b4474
##  7 GLYCOLATEMET-PWY    G7544   glycolate and glyoxylate degradation I    b4468
##  8 GLYCOLATEMET-PWY    G0-8601 glycolate and glyoxylate degradation I    b4467
##  9 GLYOXDEG-PWY        G7544   glycolate and glyoxylate degradation II   b4468
## 10 GLYOXDEG-PWY        G0-8601 glycolate and glyoxylate degradation II   b4467
## 11 LCYSDEG-PWY         G7622   L-cysteine degradation II                 b3108
## 12 LCYSDEG-PWY         G7622   L-cysteine degradation II                 b4470
## 13 SERDEG-PWY          G7624   L-serine degradation                      b3112
## 14 SERDEG-PWY          G7624   L-serine degradation                      b4471
## 15 PWY-8227            G7800   superpathway of enterobacterial common a… b4405
## 16 PWY-8227            G7800   superpathway of enterobacterial common a… b4481
## 17 GLYCOL-GLYOXDEG-PWY G7544   superpathway of glycol metabolism and de… b4468
## 18 GLYCOL-GLYOXDEG-PWY G0-8601 superpathway of glycol metabolism and de… b4467
```

It seems that, if “ecocyID” and “gene” are similar, the bnumber is not, for example: PWY0-521 and G7724, b3372 and b4474
Sometimes, bnumber and gene number are the same (b4467,G0-8601), but not the pathway. We need to check on 3 conditions: gene, bnumber, ecocyID :

```
#  Check duplicates. need remove if the bnumber, gene and ecocyID are the same
doublons22 <- pathways_to_export[duplicated(pathways_to_export[c("ecocyID", "gene","bnumber")]), ] # Vérifier si
des doublons existent
if (nrow(doublons22) > 0) {
  cat("Il y a des doublons dans le dataframe.\n")
  print(doublons22)
} else {
  cat("Aucun doublon n'a été trouvé dans le dataframe.\n")
}
```

```
## Aucun doublon n'a été trouvé dans le dataframe.
```

OK; we need to keep everything.

```
# Check na
pathways_to_export %>% filter(is.na(bnumber)) # ok (%>% length)
```

```
# Check if all the row from pathway_reshaped are in the df pathways_to_export
diff_rows <- anti_join(pathway_reshaped, pathways_to_export, by = "gene")

# Print differences
print(diff_rows)
```

```
## # A tibble: 4 × 3
##   ecocyID                   gene  desc
##   <chr>                     <chr> <chr>
## 1 ILEUSYN-PWY               G8221 L-isoleucine biosynthesis I (from threonine)
## 2 VALSYN-PWY                G8221 L-valine biosynthesis
## 3 BRANCHED-CHAIN-AA-SYN-PWY G8221 superpathway of branched chain amino acid bio…
## 4 THREOCAT-PWY              G8221 superpathway of L-threonine metabolism
```

```
# Check gene G8221
all_genes_reshaped %>% filter(gene == "G8221") #none
```

```
# so check if the corresponding ecocyID exist
all_genes_reshaped %>% filter('ecocyID' == "ILEUSYN-PWY")
```

The Genes ‘G8221’ is not present in all_genes df where the bnumber is.

Pour utiliser ces données avec python, nous allons sauvegarder le tibble au format JSON avec la library library(jsonlite) :

///# output : /## Attaching package: ‘jsonlite’ /## The following object is masked from ‘package:purrr’: /## flatten

```
# exportation
pathways_to_export %>%
  toJSON %>%
  write("../ref.sets/ecocys.pathways.sets.json")
```

Aperçu du contenu :

```
jq . "../ref.sets/ecocys.pathways.sets.json" | head -15
```

```
## [
##   {
##     "ecocyID": "PWY0-862",
##     "gene": "EG11528",
##     "desc": "(5Z)-dodecenoate biosynthesis I",
##     "bnumber": "b1288"
##   },
##   {
##     "ecocyID": "PWY0-862",
##     "gene": "EG11318",
##     "desc": "(5Z)-dodecenoate biosynthesis I",
##     "bnumber": "b1093"
##   },
##   {
##     "ecocyID": "PWY0-862",
```

EXPORT EN CSV pour neo4j

```
# pour les noeuds
pathways_to_export %>% unique %>% write_csv("../neo4j.import/ecocyc.pathways.csv")


# pour les relations connexion avec gene
pathways_to_export %>% select(c(ecocyID,bnumber)) %>% unique %>% write_csv("../neo4j.import/ecocyc.pathways.genes
.csv")
```